

Utilizando o Assembly do BIP2, escreva cada trecho de código abaixo. Utilize o BIPIDE para montar e simular.

1. Estrutura if-then

```
if (A>=B) {  
    A = A + 1 // Bloco 1  
}  
// Bloco 2
```

a < b ele pula para o L1, ele salta o bloco 1;
se não for negativo a>= b , logo ele executa bloco 1 e 2;

```
LD A ;(TRAZ A AO ACUMULADOR)  
SUB B ;(FAZ A OPERAÇÃO A - B)
```

;Se (a < b), ele vai direto para o bloco 2, pq não se adequou a regra
BLT L1 ;(REALIZA O TESTE DE QUE SE SUB B É MENOR DO QUE ZERO)

;---- BLOCO 1 ----

```
LD A ;(TRAZ A )  
ADDI 1 ; (SOMA 1 AO A)  
STO A ; (ARMAZENA, É UM =)
```

;---- BLOCO 2 --

```
L1:  
HLT
```

BIPIDE CODIGO:

```
.data  
    A    : 0  
    B    : 0  
.text  
    LD    A  
    SUB   B  
    BLT   L1  
    LD    A  
    ADDI  1  
    STO   A  
L1:  
    HLT
```

QUESTÃO 2

2. Estrutura if-then-else

```
if (A>B) {  
    A = A+1; // Bloco 1  
} else {  
    B = B+1; // Bloco 2  
}
```

```
.data  
    A:0
```

```

        B : 0
.text

        LD A
        SUB B
        BLE ELSE

THEN:
        LD A
        ADDI 1
        STO A
        JMP FIM
ELSE:

        LD B
        ADDI 1
        STO B

FIM:
HLT

```

QUESTÃO 3:

3. Estrutura while

```

i = 0;
while (i<10) {
    A += 2;      // Bloco 1
    i++;
}
// Bloco 2

```

bipide:

```

.data
    A : 0
    i : 0
.text

    LDI 0
    STO I

REPETICAO:
    SUBI 10                ; BLOCO 1
    BGE FIM (SALTA SE FOR MAIOR OU IGUAL A 10)

    LD A
    ADDI 2
    STO A

    LD I
    ADD 1
    STO I

    JMP REPETICAO

FIM:
    ; BLOCO 2

```

QUESTÃO 4:

4. Estrutura do-while

```
i = 0;
do {
    A += 2    // Bloco 1
    i++;
} while (i<10)
// Bloco 2
```

.data

```
A : 0
i : 0
```

.text

```
LDI 0
STO I
```

REPETIR: #BLOCO 1

```
LD A
ADDI 2
STO A (SEMPRE LOAD STOR, CARREGO A VARIÁVEL E GUARDO, SEMPRE)
```

```
LD I
ADDI 1
STO I
```

```
SUBI 10
BLT REPETIR
```

FIM: ;BLOCO 2

```
HLT
```

QUESTÃO 5

Código C abaixo:

```
if (a == b)
    a = a + 1;
else if (a > b)
    a = a + 2;
else if (a < b)
    a = a + 3;
```

BIPIDE

.data

```
A : 1
B : 0
```

.text

```
LD A
SUB B
BEQ EQUAL
BGT GREATER
BLT LESS
```

```
EQUAL:
    LDI 1
    JMP FIM
GREATER:
    LDI 2
    JMP FIM
LESS:
    LDI 3
FIM:
    ADD A
    STO A
    HLT
```